



Naive Bayes – Customer Churn Prediction Using SMOTE

Practical Implementation using Churn_Modelling.csv

Problem Statement

A bank wants to predict whether a customer will **churn (exit)** based on customer information.

In this practical, we will:

1. Understand the churn dataset.
2. Identify the **imbalanced target variable**.
3. Perform **One-Hot Encoding** for categorical variables.
4. Split the data into training and testing sets.
5. Apply **SMOTE only to the training data** to balance the classes.
6. Train a **Gaussian Naive Bayes** model.
7. Test and evaluate the model.
8. Compare Naive Bayes performance **before and after SMOTE**.

Practical Workflow

Data Collection → Data Preprocessing → Null Values → Visualization → Outlier Detection → X/y Separation → One-Hot Encoding → Train/Test Split → Feature Scaling → SMOTE → Naive Bayes → Testing → Evaluation → Comparison

Learning Objectives

Students will learn to:

- Build a Naive Bayes classifier for churn prediction.
- Identify class imbalance.
- Encode categorical variables using **One-Hot Encoding**.
- Understand the purpose of **SMOTE**.
- Apply SMOTE correctly without data leakage.
- Evaluate an imbalanced classification model using Accuracy, Precision, Recall, F1-score and Confusion Matrix.

1. Data Collection

Dataset: `Churn_Modelling.csv`

Target Variable

`Exited`

- `0` → Customer stayed
- `1` → Customer churned

Important Features

`CreditScore`, `Geography`, `Gender`, `Age`, `Tenure`, `Balance`,
`NumOfProducts`, `HasCrCard`, `IsActiveMember`, and `EstimatedSalary`.

Keep `Churn_Modelling.csv` in the same folder as this notebook.

```
In [119... # 1.1 Import Libraries and Dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)

from imblearn.over_sampling import SMOTE

# Load dataset
df = pd.read_csv("Churn_Modelling.csv")

print("Dataset Shape:", df.shape)
df.head()
```

Dataset Shape: (10000, 14)

Out[119...	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	T
0	1	15634602	Hargrave	619	France	Female	42.0	
1	2	15647311	Hill	608	Spain	Female	41.0	
2	3	15619304	Onio	502	France	Female	42.0	
3	4	15701354	Boni	699	France	Female	39.0	
4	5	15737888	Mitchell	850	Spain	Female	43.0	

1.2 Checking for Null Values

Before building the model, check for missing values.

For this dataset:

- `Age` is numerical → fill missing values with the **median**.
- `Gender` is categorical → fill missing values with the **mode**.

```
In [122... print(df.isnull().sum())
```

```
RowNumber      0
CustomerId     0
Surname        0
CreditScore    0
Geography      0
Gender         3
Age            3
Tenure         0
Balance        0
NumOfProducts  0
HasCrCard      0
IsActiveMember 0
EstimatedSalary 0
Exited         0
dtype: int64
```

```
In [124... # Handle missing values
```

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

print("Missing values after treatment:")
print(df.isnull().sum())
```

```
Missing values after treatment:
RowNumber      0
CustomerId     0
Surname        0
CreditScore    0
Geography      0
Gender         0
Age           0
Tenure        0
Balance       0
NumOfProducts 0
HasCrCard     0
IsActiveMember 0
EstimatedSalary 0
Exited        0
dtype: int64
```

1.3 Data Visualization – Check Class Imbalance

The target variable is `Exited`.

- `0` → Stayed
- `1` → Churned

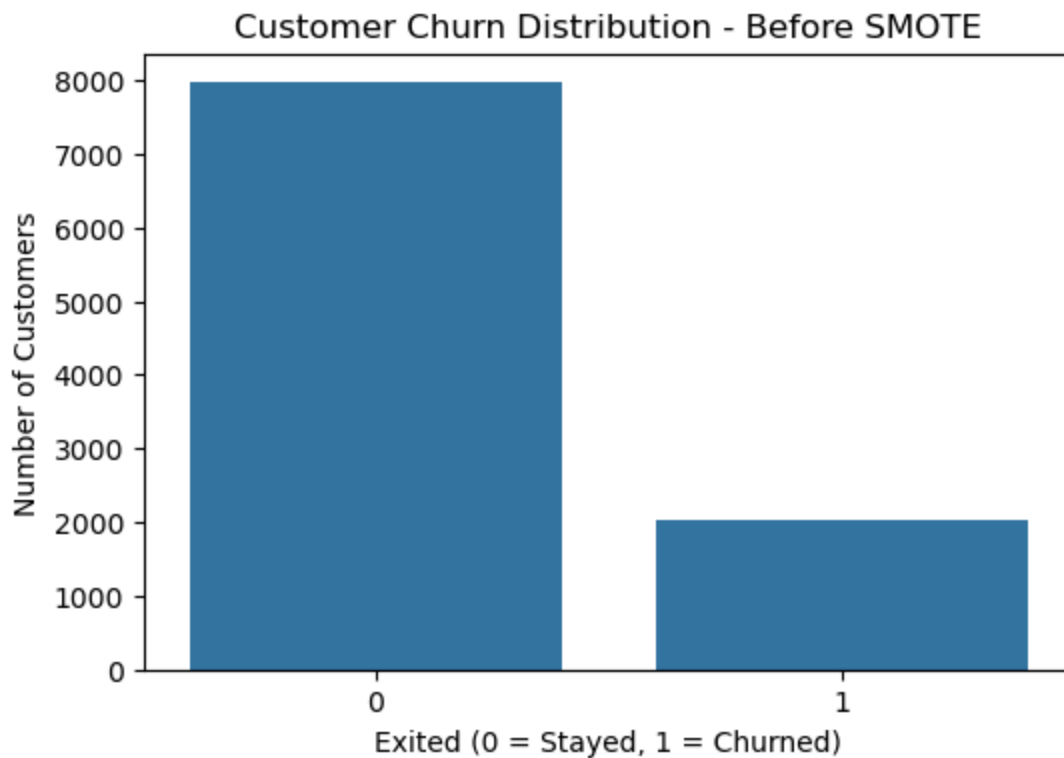
Let us visualize the distribution.

If one class has many more observations than the other, the dataset is **imbalanced**.

```
In [127... print(df['Exited'].value_counts())

plt.figure(figsize=(6,4))
sns.countplot(x='Exited', data=df)
plt.title("Customer Churn Distribution - Before SMOTE")
plt.xlabel("Exited (0 = Stayed, 1 = Churned)")
plt.ylabel("Number of Customers")
plt.show()
```

```
Exited
0      7963
1      2037
Name: count, dtype: int64
```



Why is Class Imbalance a Problem?

Suppose 80% of customers stayed and 20% churned.

A model that predicts **Stay for everyone** can achieve 80% accuracy, but it identifies **none of the churned customers**.

Therefore, for an imbalanced classification problem:

Accuracy alone is not enough.

We should also examine **Precision, Recall, F1-score and the Confusion Matrix**, especially for the churn class.

1.4 Outlier Detection

We inspect important numerical features using boxplots.

We do not automatically remove every outlier because an extreme customer value may be a genuine observation.

```
In [130... numeric_columns = [  
    'CreditScore',  
    'Age',
```

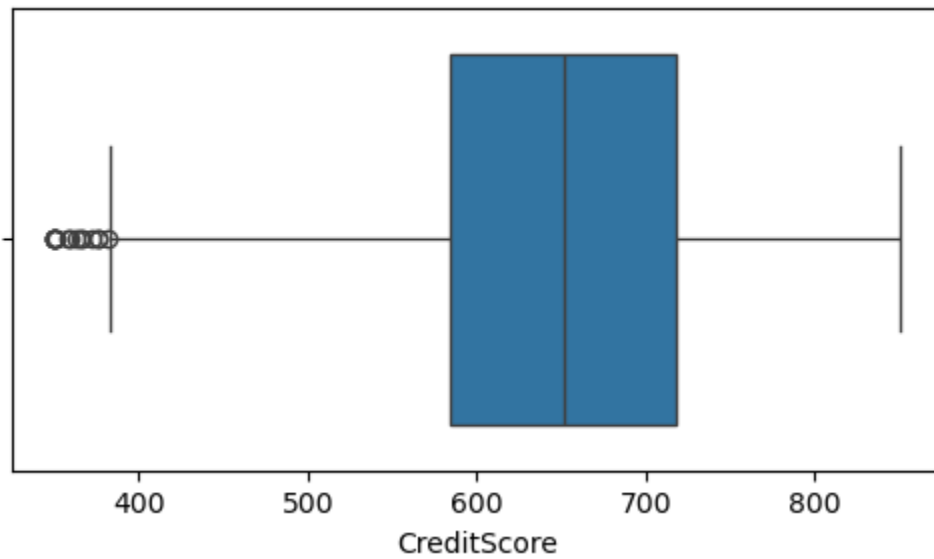
```

'Tenure',
'Balance',
'NumOfProducts',
'EstimatedSalary'
]

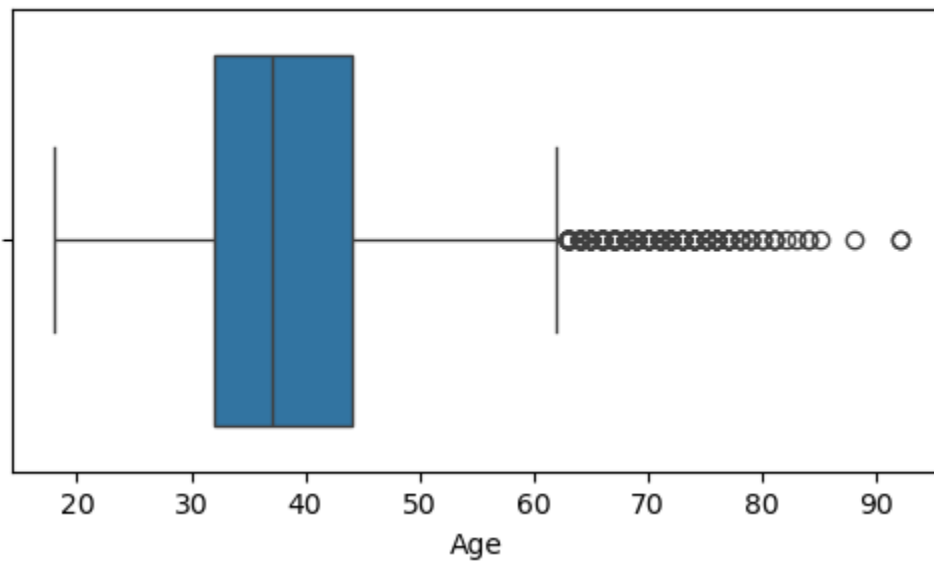
for column in numeric_columns:
    plt.figure(figsize=(6,3))
    sns.boxplot(x=df[column])
    plt.title(f"Boxplot of {column}")
    plt.show()

```

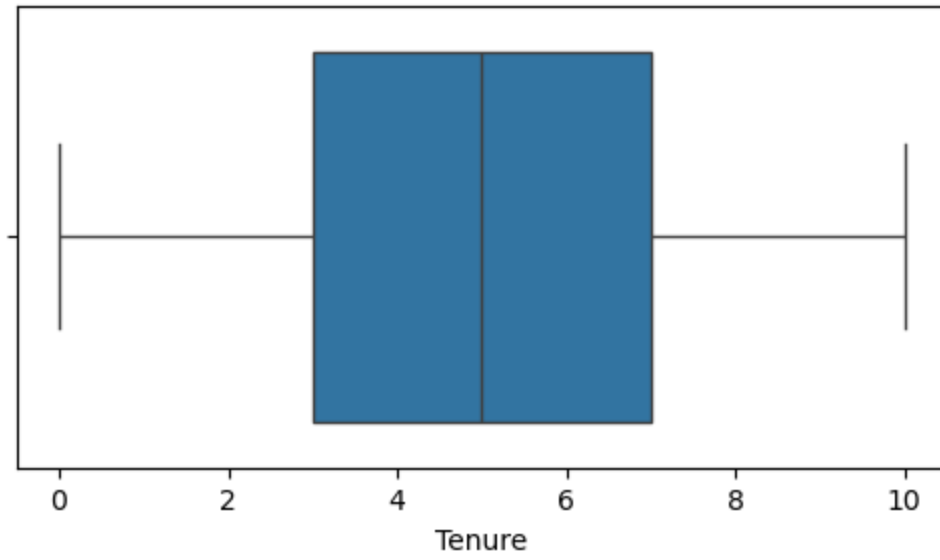
Boxplot of CreditScore



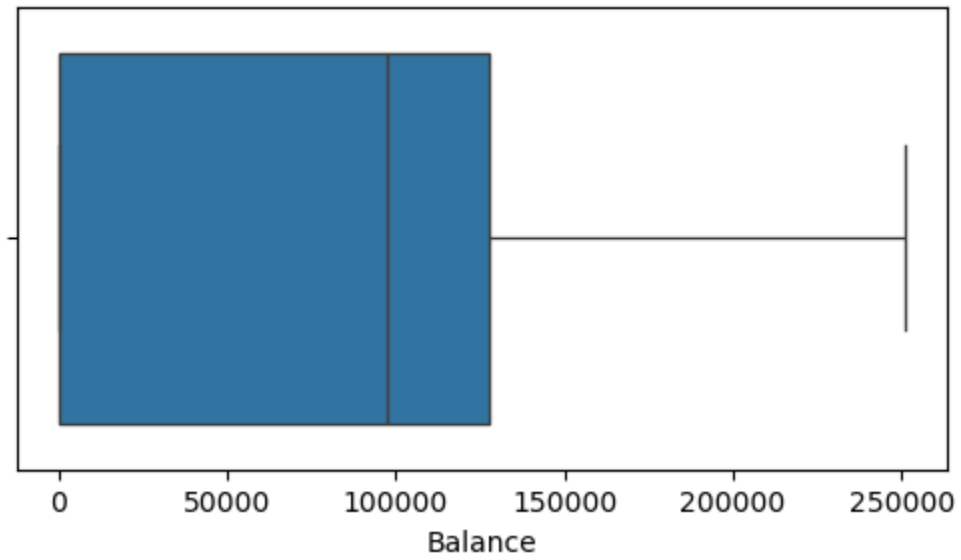
Boxplot of Age

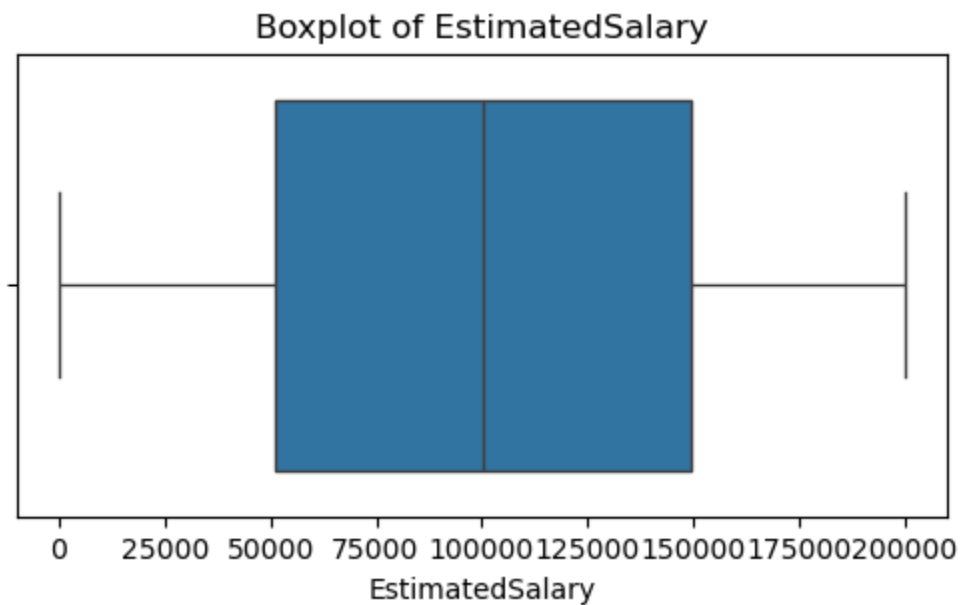
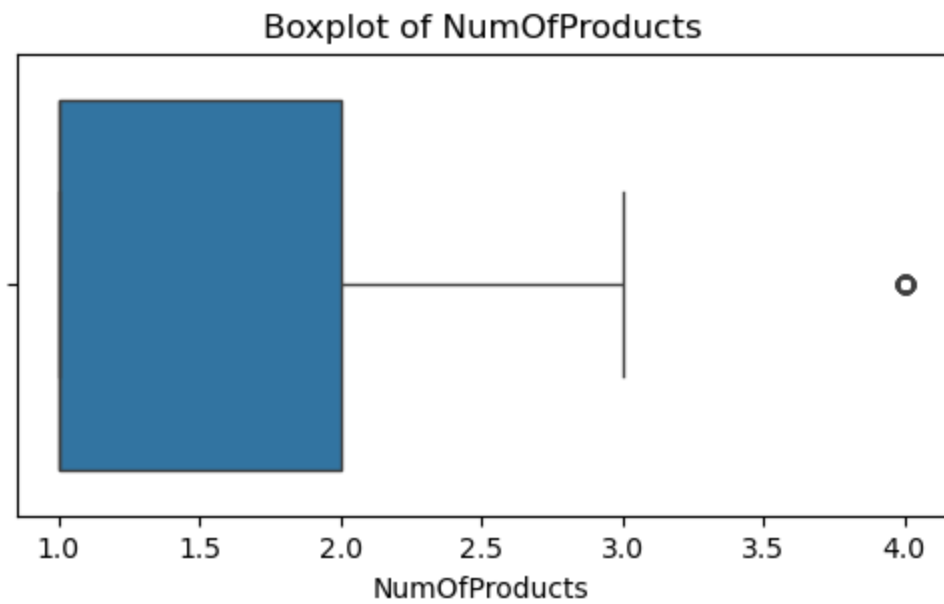


Boxplot of Tenure



Boxplot of Balance





1.5 Splitting Independent and Dependent Variables

The target variable is `Exited`.

We remove identifier columns:

- `RowNumber`
- `CustomerId`
- `Surname`

These identify customers but do not provide useful predictive information for this model.

```
In [133... X = df.drop(
    ['RowNumber', 'CustomerId', 'Surname', 'Exited'],
    axis=1
)
y = df['Exited']

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (10000, 10)
y shape: (10000,)
```

1.6 One-Hot Encoding

The categorical variables are:

- Geography
- Gender

We will perform **One-Hot Encoding before the train-test split**, as requested.

For example, `Geography` is converted into numerical indicator columns such as:

- `Geography_Germany`
- `Geography_Spain`

and `Gender` becomes a numerical indicator column.

`pd.get_dummies()` only changes the representation of the categorical values; it does not calculate statistics such as mean or standard deviation. The actual learned preprocessing steps such as scaling and SMOTE will still be performed only after the train-test split.

```
In [135... # One-Hot Encoding BEFORE Train-Test Split

X = pd.get_dummies(
    X,
    columns=['Geography', 'Gender'],
    drop_first=True
)

print("Shape after One-Hot Encoding:", X.shape)
X.head()
```

```
Shape after One-Hot Encoding: (10000, 11)
```

Out[135...

	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMe
0	619	42.0	2	0.00	1	1	
1	608	41.0	1	83807.86	1	0	
2	502	42.0	8	159660.80	3	1	
3	699	39.0	1	0.00	2	0	
4	850	43.0	2	125510.82	1	1	

1.7 Train-Test Split

Now that the categorical variables have been converted into numerical columns, split the data into training and testing sets.

We use:

- **80% Training data**
- **20% Testing data**

`stratify=y` maintains a similar proportion of churned and non-churned customers in both sets.

Important

We still **do not apply SMOTE before splitting**.

Correct order:

One-Hot Encoding → Train/Test Split → Scaling → SMOTE on Training Data → Model Training

```
In [137... X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)

print("\nTraining class distribution:")
print(y_train.value_counts())
```

```
print("\nTesting class distribution:")
print(y_test.value_counts())
```

Training data: (8000, 11)

Testing data : (2000, 11)

Training class distribution:

Exited

0 6370

1 1630

Name: count, dtype: int64

Testing class distribution:

Exited

0 1593

1 407

Name: count, dtype: int64

1.8 Feature Scaling

Naive Bayes is **not a distance-based algorithm**, so scaling is not required in the same way it is for KNN.

However, we use standardization here to create a consistent numerical feature space before applying SMOTE.

The important rule is:

Fit the scaler only on the training data and use that fitted scaler to transform the test data.

```
In [143... scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training data scaled.")
print("Testing data scaled.")
```

Training data scaled.

Testing data scaled.

1.9 Apply SMOTE to the Training Data

What is SMOTE?

SMOTE = Synthetic Minority Over-sampling Technique

SMOTE creates synthetic observations for the minority class rather than simply duplicating existing observations.

Most Important Rule

Apply SMOTE only to the training data. Never apply SMOTE to the test data.

Correct:

One-Hot Encoding → Train/Test Split → Scale → SMOTE on Training Data → Train Model → Test on Untouched Test Data

This prevents data leakage.

The training data can be modified because it is used to teach the model. The test data should remain untouched because it is used to check how the model performs on real, unseen data.

```
In [146... from imblearn.over_sampling import SMOTE

print("Before SMOTE:")
print(y_train.value_counts())

smote = SMOTE(random_state=42)

X_train_balanced, y_train_balanced = smote.fit_resample(
    X_train_scaled,
    y_train
)

print("\nAfter SMOTE:")
print(y_train_balanced.value_counts())
```

Before SMOTE:

Exited

0 6370

1 1630

Name: count, dtype: int64

After SMOTE:

Exited

1 6370

0 6370

Name: count, dtype: int64

In []:

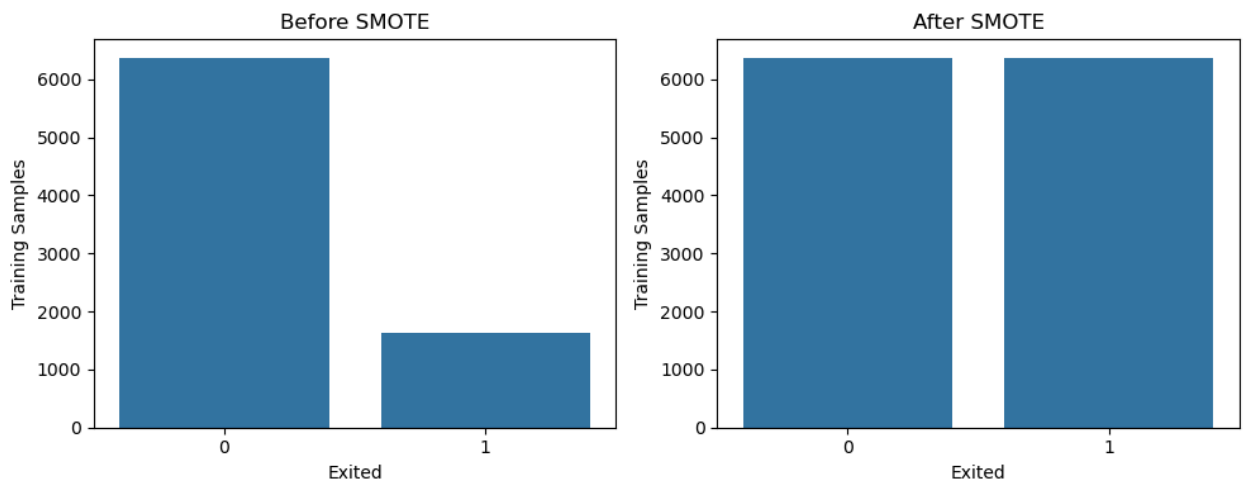
```
In [148... # Visualize class distribution before and after SMOTE
```

```
fig, axes = plt.subplots(1, 2, figsize=(10,4))

sns.countplot(x=y_train, ax=axes[0])
axes[0].set_title("Before SMOTE")
axes[0].set_xlabel("Exited")
axes[0].set_ylabel("Training Samples")

sns.countplot(x=y_train_balanced, ax=axes[1])
axes[1].set_title("After SMOTE")
axes[1].set_xlabel("Exited")
axes[1].set_ylabel("Training Samples")

plt.tight_layout()
plt.show()
```



What happened after SMOTE?

Before SMOTE, the churn class (1) is the minority class.

After SMOTE, synthetic minority-class observations are created so the training classes become balanced.

The test data remains completely untouched.

Because ordinary SMOTE is being applied after one-hot encoding, synthetic values can be interpolated in the encoded categorical columns. For mixed numerical/categorical production data, **SMOTENC** is often preferable.

What happened after SMOTE?

Before SMOTE, the churn class (1) is the minority class.

After SMOTE, new synthetic minority-class observations are created so that the training classes are balanced.

The test data has not been changed.

Important Note

Because we are using ordinary SMOTE after one-hot encoding, synthetic observations can contain interpolated values in the encoded categorical columns. This is a simple teaching workflow.

For production work with mixed numerical/categorical data, **SMOTENC** is often preferable.

2. Model Building – Gaussian Naive Bayes

Naive Bayes is a **probabilistic supervised learning algorithm** based on Bayes' theorem.

For this churn problem, we use **GaussianNB**, which assumes continuous features follow a Gaussian distribution within each class.

The model calculates the probability of each class and predicts the class with the highest posterior probability.

```
In [153]: # Initialize and train Gaussian Naive Bayes

nb_model = GaussianNB()

nb_model.fit(
    X_train_balanced,
    y_train_balanced
)

print("Gaussian Naive Bayes model trained successfully.")
```

Gaussian Naive Bayes model trained successfully.

3. Testing the Model

We now predict churn on the **original untouched test data**.

- Training data → balanced using SMOTE.

- Test data → original distribution.

```
In [156... y_pred = nb_model.predict(X_test_scaled)

print("First 20 predictions:")
print(y_pred[:20])

print("\nFirst 20 actual values:")
print(y_test.values[:20])
```

```
First 20 predictions:
[0 1 0 0 0 0 0 1 1 0 1 0 0 0 0 0 0 0 1 1]
```

```
First 20 actual values:
[0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0]
```

4. Evaluation of the Model

We evaluate the model using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Why is Recall important for Churn?

Recall answers:

Of all the customers who actually churned, how many did the model correctly identify?

A false negative means:

Actual = Churn, Predicted = Stay

This can represent a missed customer-retention opportunity.

```
In [159... print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Accuracy : 0.7135
Precision: 0.3850415512465374
Recall : 0.683046683046683
F1 Score : 0.4924712134632418

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.72	0.80	1593
1	0.39	0.68	0.49	407
accuracy			0.71	2000
macro avg	0.64	0.70	0.65	2000
weighted avg	0.79	0.71	0.74	2000

In [161]: *# Confusion Matrix*

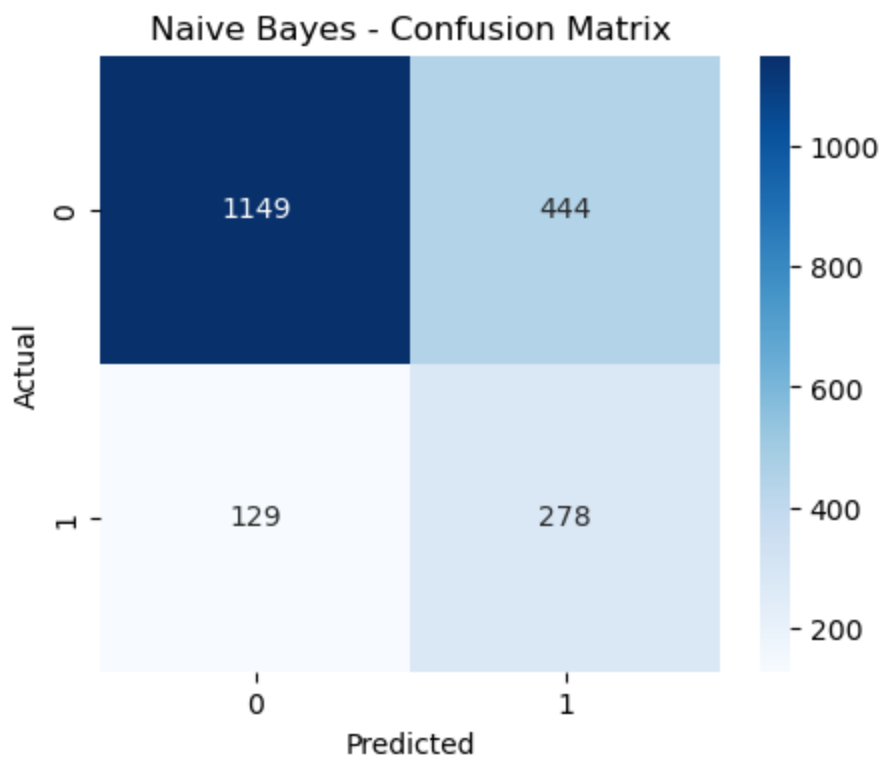
```
cm = confusion_matrix(y_test, y_pred)

print(cm)

plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Naive Bayes - Confusion Matrix")
plt.show()
```

```
[[1149  444]
 [ 129  278]]
```

5. Compare Naive Bayes Before and After SMOTE

To understand the effect of balancing, we train a second Naive Bayes model using the **original imbalanced training data**.

Then we compare it with the model trained after SMOTE.

The key question is:

Did balancing improve the model's ability to identify churned customers?

Do not judge the result only by accuracy. Pay particular attention to **Recall and F1-score for the churn class**.

```
In [164... # Train a baseline Naive Bayes model on the original imbalanced training data
nb_baseline = GaussianNB()
nb_baseline.fit(
    X_train_scaled,
    y_train
)
```

```

y_pred_baseline = nb_baseline.predict(X_test_scaled)

print("Naive Bayes - Before SMOTE")
print(classification_report(y_test, y_pred_baseline))

```

```

Naive Bayes - Before SMOTE
              precision    recall  f1-score   support

    0       0.85         0.94         0.89        1593
    1       0.60         0.35         0.44         407

 accuracy         0.82         0.82         0.82        2000
 macro avg       0.72         0.64         0.67        2000
weighted avg       0.80         0.82         0.80        2000

```

In [166... *# Compare Before vs After SMOTE*

```

comparison = pd.DataFrame({
    'Model': [
        'Naive Bayes - Before SMOTE',
        'Naive Bayes - After SMOTE'
    ],
    'Accuracy': [
        accuracy_score(y_test, y_pred_baseline),
        accuracy_score(y_test, y_pred)
    ],
    'Precision (Churn)': [
        precision_score(y_test, y_pred_baseline),
        precision_score(y_test, y_pred)
    ],
    'Recall (Churn)': [
        recall_score(y_test, y_pred_baseline),
        recall_score(y_test, y_pred)
    ],
    'F1 (Churn)': [
        f1_score(y_test, y_pred_baseline),
        f1_score(y_test, y_pred)
    ]
})

comparison

```

Out[166...

	Model	Accuracy	Precision (Churn)	Recall (Churn)	F1 (Churn)
0	Naive Bayes - Before SMOTE	0.8200	0.600000	0.346437	0.439252
1	Naive Bayes - After SMOTE	0.7135	0.385042	0.683047	0.492471

Interpretation of Results

Model	Accuracy	Precision (Churn)	Recall (Churn)	F1 (Churn)
Naive Bayes – Before SMOTE	82.00%	60.00%	34.64%	43.93%
Naive Bayes – After SMOTE	71.35%	38.50%	68.30%	49.25%

Interpretation

Before SMOTE:

The model achieves a higher accuracy of **82%**, but the recall for churn is only **34.64%**. This means the model is missing many customers who actually churn.

After SMOTE:

The accuracy decreases to **71.35%**, but the recall for churn increases significantly from **34.64% to 68.30%**. This means the model is now able to identify many more actual churn customers.

Key Observation

SMOTE trades some overall accuracy for much better detection of the minority class (churn).

The F1-score for churn also improves from **43.93% to 49.25%**, indicating a better balance between precision and recall.

Conclusion

For a **churn prediction problem**, the model after SMOTE may be more useful because identifying customers who are likely to churn can be more important than achieving the highest overall accuracy.

Therefore, we should not evaluate an imbalanced classification problem using accuracy alone. Recall and F1-score for the minority class should also be considered.

Understanding SMOTE – Simple Explanation

SMOTE does not simply copy minority-class records.

It:

1. Selects a minority-class observation.
2. Finds nearby minority-class observations.
3. Creates a synthetic observation between them.
4. Repeats the process until the desired balance is reached.

Easy way to remember

Imbalanced data → Minority class is too small → SMOTE creates synthetic minority samples → Balanced training data → Model gets more information about the minority class

Important

Never balance the test set.

The test set should represent the original distribution of customers.

Final Takeaways

1. **Naive Bayes is a probabilistic supervised classification algorithm.**
2. The Churn Modelling dataset is **imbalanced**.
3. **Geography** and **Gender** are categorical variables.
4. We use **One-Hot Encoding** to convert categorical variables into numerical features.
5. We apply **SMOTE only to the training data**.
6. The test data remains untouched.
7. Accuracy alone is not sufficient for an imbalanced classification problem.
8. Examine **Precision, Recall and F1-score**, especially for **Exited = 1**.
9. Compare the model before and after SMOTE.
10. SMOTE is a technique for balancing the training data; it is not a

machine learning model.

In []: